

DOC_INTEGRITY_INTEGRATION

DOC_INTEGRITY_INTEGRATION.md — Wiring the §11.4.186 Anti-Divergence Engine into HelixCode’s Export Pipeline

Field	Value
Revision	1
Created	2026-07-08
Last modified	2026-07-08T12:00:00Z
Status	active
Status summary	Design document for integrating the constitution submodule’s doc-integrity validator into HelixCode’s three doc-export/commit seams.
Authority	Constitution §11.4.186 (anti-divergence enforcement) + §11.4.106 (Docs Chain) + §11.4.65 (universal markdown export)
Author	T1/main conductor

1. Purpose

This document describes how to wire the **doc-integrity** Go engine (constitution/scripts/doc_integrity/) into HelixCode’s three write-seams — export render, doc/DB sync verify, and doc-set commit — so that cross-document CONSISTENCY is enforced as a deterministic PASS/FAIL/SKIP gate that runs BEFORE any of those seams proceeds, as mandated by Constitution §11.4.186.

2. Engine overview (what already exists)

A project-agnostic (§11.4.28) Go tool residing at <repo-root>/constitution/scripts/doc_integrity/, consumed **by reference** per §11.4.177 / §11.4.80-style inheritance (NEVER copied into HelixCode).

Property	Value
Module	github.com/HelixDevelopment/ HelixConstitution/scripts/ doc_integrity
Go version	1.25
Stack	excelize (xlsx), go-sqlite3, yaml.v3
Entry	cmd/doc_integrity/main.go — subcommands: verify, report, selfcheck, version
Exit codes	0 PASS, 1 FAIL, 2 config-error, 3 SKIP (source unavailable, honest §11.4.3)

2.1 Five check families

Family	Rules	Action on FAIL
DEDUP	DEDUP-01/02	Duplicate feature descriptions with divergent timeline/status/release. Keyed on ticket OR normalised (subject, scope) — NOT bare subject substring (false-positive guard for distinct same-subject tasks, §11.4.6).
TIMELINE	TIME-01/01b/ 02/03/04	start ≤ deadline; no deadline-before-dependency; no DAG cycles; section span containment; GATED exempt-but-marked.
CROSS-DOC	XDOC-01	The SAME feature across plan/MVP/summary/DB

Family	Rules	Action on FAIL
		(joined by ticket, fuzzy-subject fallback) carries identical timeline/status/type against a NAMED authoritative source.
INTEGRITY	INTEG-01/02/03	No orphan refs; Status-Type agreement (§11.4.33 Bug → Fixed / Feature → Implemented / Task → Completed); location ↔ status agreement.
STRUCTURAL	STRUCT-01/02	Required columns present; item IDs unique + match id_pattern (+ opt-in monotonic).

2.2 Four adapters (source kinds)

- **xlsx** (excelize) — for .xlsx plan files
- **markdown-table** — for markdown pipe tables (Issues_Summary.md style)
- **markdown-headings** — for Issues/Fixed.md H2-format records
- **sqlite** (READ-ONLY) — for docs/workable_items.db

2.3 Anti-bluff self-validation

The selfcheck subcommand runs embedded golden fixtures under internal/selfcheck/golden/:

- **golden-good**: MUST PASS (validator does not false-positive)
- **golden-bad (one per family)**: MUST FAIL, pinpointing the injected offender (validator catches real defects)
- **negative-control**: distinct same-subject tasks that MUST PASS (validator does not false-positive on (subject, scope) distinction)

A validator that PASSES a golden-bad fixture, or FAILs golden-good / the negative-control, is itself a §11.4 bluff and the gate's meta-test catches it.

2.4 Gate hooks

1. **wire/doc_integrity_gate.sh <checkset.yaml> [repo_root] [--divergence-class-only]** — the HARD gate hook. Called BEFORE any

render in `sync_all_markdown_exports.sh / Docs Chain verify / commit_all.sh`; export/commit is REFUSED on exit 1.

2. **wire/CM-DOC-INTEGRITY-VALIDATION.gate.sh** — recommended pre-build gate stub with 5 invariants (i)–(v).

The `--divergence-class-only` flag splits the finding families at §11.4.186 clause-6 honest boundary: INTEGRITY-class findings (orphan-ref / Status-Type / location-status DATA defects) become NON-refusing — plan-data correctness is an operator decision the gate SURFACES, never MAKES. Only DEDUP / TIMELINE / CROSS-DOC / STRUCTURAL still REFUSE. This is the recommended initial posture until plan data is cleaned up (§11.4.50 ratchet: strengthen monotonically over time).

3. HelixCode's export/document landscape

3.1 Export pipeline

The canonical markdown export pipeline is at:

`<repo-root>/scripts/testing/sync_all_markdown_exports.sh`

This is a **meta-repo-root** script that renders all `docs/**/* .md` to `.html + .pdf` siblings via `pandoc + weasyprint`, with optional Mermaid-aware preprocessing (`mmdc`). It serves the ENTIRE meta-repo's doc set (constitution docs, `helix_code` docs, `heliqa` docs, etc.).

Key characteristics: - Mode: `--check-only`, `--regenerate-all`, `--file <path>`
- Idempotent: stale-sibling detection by `mtime` - Mermaid-awareness: `effective_source()` preprocessor renders each ````mermaid` fence to a PNG before `pandoc` sees the doc (§11.4.168) - Invoked from the meta-repo root

3.2 HelixCode doc-set (what the checkset would register)

The following docs under `<repo-root>` carry tracked data that could diverge across representations:

Document	Format	Adapter	Role
<code>docs/research/07.2026/00_master/MASTER_IMPLEMENTATION_PLAN.md</code>	markdown (plan-style tables)	markdown-table + markdown-headings	HelixCode's delivery plan
<code>docs/Issues.md</code>	markdown H2 entries	markdown-headings	Open work items

Document	Format	Adapter	Role
docs/Issues_Summary.md	markdown table	markdown-table	Open-item summary
docs/Fixed.md	markdown H2 entries	markdown-headings	Closed work items
docs/Fixed_Summary.md	markdown table	markdown-table	Closed-item summary
docs/workable_items.db	SQLite	sqlite (READ-ONLY)	Workable-items SSoT (§11.4.93/.95)

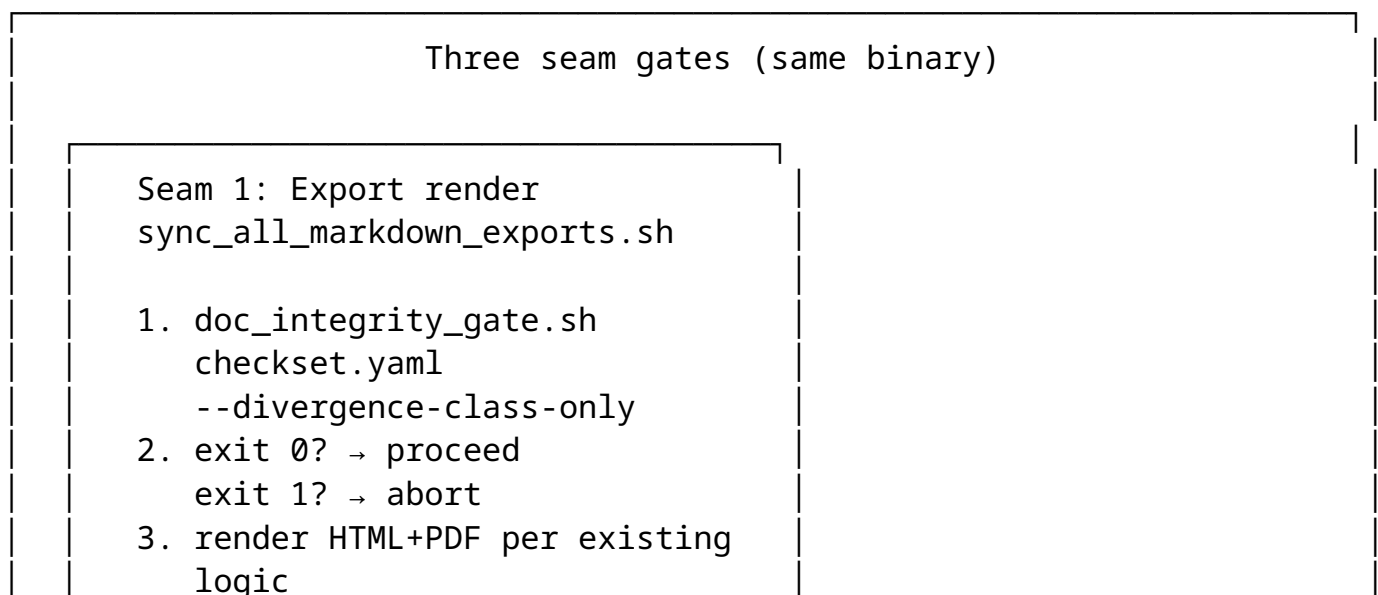
The checkset registers these sources and their authoritative bindings (e.g., `workable_items.db` is authoritative for status/type, plan for timeline).

3.3 Three seams that must be gated

Per §11.4.186 clause 1, the single no-divergence verdict gates ALL THREE:

1. **Export seam** — `scripts/testing/sync_all_markdown_exports.sh` (render HTML+PDF). Gate runs FIRST, before any render.
2. **Doc/DB sync verify** — Docs Chain verify command (§11.4.106) — register a `doc_integrity` check node in the consumer's `.docs_chain/contexts/*.yaml`.
3. **Doc-set commit** — `scripts/commit_all.sh` or `scripts/commit_docs.sh` (§11.4.22). Refuse the doc-set commit class on FAIL.

4. Integration design



```
Seam 2: Docs Chain verify
docs_chain verify <context>
```

```
New check node: doc_integrity
→ doc-integrity verify checkset.yaml
```

```
Seam 3: Doc-set commit
commit_all.sh / commit_docs.sh
```

```
Pre-flight: doc_integrity_gate.sh
checkset.yaml (full strict mode)
exit 0? → commit proceeds
exit 1? → commit refused
```

4.1 Seam 1: Export render gate

Insert into scripts/testing/sync_all_markdown_exports.sh BEFORE the loop that renders siblings.

Change: At line ~60, after variable setup and before collect the .md source set:

```
# --- §11.4.186 doc-integrity gate (anti-divergence enforcement)
---
# Runs BEFORE any export render – a FAIL here refuses the export.
CHECKSET="$REPO_ROOT/.helix_code/doc_integrity/checkset.yaml"
if [ -f "$CHECKSET" ] && command -v bash >/dev/null 2>&1; then
    if ! bash constitution/scripts/doc_integrity/wire/
        doc_integrity_gate.sh \
            "$CHECKSET" "$REPO_ROOT" --divergence-class-only; then
        # exit 1 means hard FAIL; exit 3 means SKIP (source
        # unavailable – proceed
        # with warning). Only exit 1 aborts.
        rc=$?
        if [ "$rc" -eq 1 ]; then
```

```

    echo
    "FATAL §11.4.186: doc-integrity FAIL – export REFUSED.
    Fix cross-doc" \
    "divergences before exporting." >&2
    exit 1
fi
fi
else
    echo "WARN §11.4.186/§11.4.3: checkset not found at
    $CHECKSET – gate skipped" >&2
fi
# --- end doc-integrity gate ---

```

The `--divergence-class-only` flag is used for the export seam (tolerates INTEGRITY-class plan-data correctness issues; only catches genuine DEDUP/TIMELINE/CROSS-DOC/STRUCTURAL divergences). The commit seam (Seam 3) uses full-strict mode (no flag) so no integrity issue can land.

4.2 Seam 2: Docs Chain verify

Register a `doc_integrity` check node in the consumer's Docs Chain context at `.docs_chain/contexts/doc_integrity.yaml`:

```

# .docs_chain/contexts/doc_integrity.yaml
# §11.4.186 check node – cross-document consistency gate
checks:
  - name: doc_integrity
    command: |
      constitution/scripts/doc_integrity/wire/
      doc_integrity_gate.sh \
      .helix_code/doc_integrity/checkset.yaml "$REPO_ROOT" \
      --divergence-class-only
    description: >
      Run the doc-integrity validator against the registered doc-
      set.
      REFUSES on any DEDUP/TIMELINE/CROSS-DOC/STRUCTURAL finding.
      INTEGRITY-class findings are surfaced but non-refusing
      (--divergence-class-only).
    exit_on_fail: true

```

4.3 Seam 3: Doc-set commit gate

Insert into `scripts/commit_all.sh` (or a delegated `scripts/commit_docs.sh` per §11.4.22) BEFORE the staging step, using FULL strict mode (no `--divergence-class-only`):

```
# §11.4.186 pre-commit doc-integrity gate (full strict)
CHECKSET="$REPO_ROOT/.helix_code/doc_integrity/checkset.yaml"
if [ -f "$CHECKSET" ]; then
    if ! bash constitution/scripts/doc_integrity/wire/
        doc_integrity_gate.sh \
        "$CHECKSET" "$REPO_ROOT"; then
        rc=$?
        if [ "$rc" -eq 1 ]; then
            echo "FATAL §11.4.186: doc-integrity FAIL – commit
                REFUSED." >&2
            exit 1
        fi
    fi
fi
```

5. Consumer checkset

The checkset is the **consumer-owned** configuration file (§11.4.28, decoupled from the engine) at <repo-root>/.helix_code/doc_integrity/checkset.yaml. It registers HelixCode's doc-set and the authoritative-source bindings.

5.1 Schema reference

Full schema documented in the engine's design at constitution/scripts/doc_integrity/DESIGN.md §1.4. Key elements:

```
sources:
- id: "workable_items_db"
  kind: sqlite
  path: "docs/workable_items.db"
  query: "SELECT atm_id AS ticket, type, status, subject_norm,
    date(created) AS start FROM items"
- id: "plan_v9_xlsx"
  kind: xlsx
  path: "docs/research/07.2026/00_master/plan.xlsx" # if
    maintained
  sheet: "Plan"
  columns:
    ticket: "A"
    subject: "B"
    start: "C"
    deadline: "D"
    status: "E"
    type: "F"
```

```

crossdoc_join:
  authoritative: "workable_items_db" # DB is the SSoT per
    §11.4.95
  compare: ["status", "type"] # timeline comparison opt-
    in when plans are maintained
  fallback: "subject_norm"
thresholds:
  subject_similarity: 0.80
id_pattern: "^ATM-\\d+$"
check_families:
  - DEDUP
  - TIMELINE
  - CROSS-DOC
  - INTEGRITY
  - STRUCTURAL

```

5.2 Initial posture (phased ratchet per §11.4.50)

Phase	Condition	Flag	What it enforces
1 (initial)	Seam 1 (export)	--divergence-class-only	DEDUP/TIMELINE/CROSS-DOC/STRUCTURAL only. Plan-data correctness errors surfaced but non-blocking.
2 (ratchet)	Seam 3 (commit)	(no flag, full strict)	ALL five families including INTEGRITY. No integrity finding can land in a commit.
3 (future)	Seam 1 (export)	(drop --divergence-class-only)	Full strict on export too — once plan data is clean, strengthen monotonically.

6. Pre-build gate

The existing `CM-DOC-INTEGRITY-VALIDATION.gate.sh` stub (at `constitution/scripts/doc_integrity/wire/`) asserts the 5 DESIGN §5.2 invariants:

Invariant	What it checks	Status
(i)	<code>doc_integrity</code> module exists AND builds	READY (module on disk, go build ./... proven)
(ii)	<code>selfcheck</code> exits 0 (golden fixtures wired)	READY (embedded golden/good/bad/negative-control)
(iii)	<code>export seam</code> invokes the gate	PENDING — requires the Seam 1 insertion from §4.1
(iv)	consumer checkset exists and parses	PENDING — requires authoring the checkset from §5
(v)	<code>verify over the LIVE doc-set</code> exits 0	PENDING — depends on (iv) + the live doc-set being clean

The gate stub reports (iii)+(iv)+(v) as **PENDING** (honest, §11.4.6) until the checkset and seam edits land. Once all five pass, the gate goes green.

7. Anti-bluff self-validation for HelixCode's integration

Per §11.4.107(10) and §11.4.186 clause 4:

1. The **engine** is self-validated by its own `selfcheck` (golden-good PASS, golden-bad per family MUST FAIL, negative-control PASS). This runs as invariant (ii) of the pre-build gate.
2. The **integration** must be mutation-tested independently. A paired §1.1 mutation at the Seam-1 level: temporarily remove the `doc_integrity_gate.sh` invocation from `sync_all_markdown_exports.sh` and assert the gate's invariant (iii) FAILs. Revert after assertion.
3. The **checkset** must be validated: `doc-integrity verify <checkset>` with a deliberately corrupted checkset path must exit 2 (config-error) —

never a fake PASS. This is covered by the gate's own behaviour (missing checkset → exit 3 SKIP, malformed → exit 2).

8. Composition with existing mandates

Mandate	How they compose
§11.4.65 / CONST-066 (universal markdown export)	The export pipeline already exists. The doc-integrity gate runs BEFORE the render loop — it does NOT replace any check, it ADDS a pre-flight consistency check.
§11.4.168 (exported-doc visual validation)	Independent validator for the output artifact (PDF/HTML). Doc-integrity validates INPUT consistency. Both compose: the source must be internally consistent (doc-integrity) AND the output must be readable (visual validation). Neither substitutes for the other.
§11.4.106 (Docs Chain)	Doc-integrity becomes a new check node in the Docs Chain verify, registered via the consumer's <code>.docs_chain/contexts/*.yaml</code> .
§11.4.108 (four-layer fix-verification)	Doc-integrity is a SOURCE-layer gate (pre-export/commit), complementing the ARTIFACT (post-build) and RUNTIME-ON-CLEAN-TARGET (post-deploy) layers.
§11.4.86 (roster-backed auto-sync)	The checkset fingerprint (sha256 of sorted member key set) triggers re-verification on ANY input change. The fingerprint check runs as part of the gate invocation.

9. Implementation order

#	Step	Dependencies	Effort
1			Low

#	Step	Dependencies	Effort
	Author <code>.helix_code/doc_integrity/checkset.yaml</code> registering the doc-set from §3.2	Must know which docs are tracked	
2	Insert gate into <code>scripts/testing/sync_all_markdown_exports.sh</code> per §4.1	Checkset exists	Low
3	Register the <code>doc_integrity</code> check node in <code>.docs_chain/contexts/</code> per §4.2	Docs Chain context directory exists	Low
4	Insert gate into commit wrapper per §4.3	Checkset exists	Low
5	Wire the pre-build gate stub into the project's pre-build verification script	Steps 1-2 done	Low
6	Author the paired §1.1 mutation (strip gate from export seam → invariant FAILs)	Steps 2 done	Low
7	Run <code>doc-integrity verify <checkset></code> against the live doc-set, fix any surfaced divergences	Steps 1 done (data cleanup may be iterative)	Medium
8	Ratchet: when plan data is clean, drop <code>--divergence-class-only</code> from Seam 1	Step 7 done	Low

10. Overview

The doc-integrity engine is **already built** and **self-validated**. What remains for HelixCode is consumer-side configuration and seam wiring — the YAML checkset, three Bash-level gate insertions, and one pre-build gate entry. The engine's exit behaviour (0 PASS / 1 FAIL / 2 config-error / 3 SKIP) is designed for exactly this kind of non-invasive integration: the gate hook handles toolchain absence gracefully (exit 3 SKIP) and the `--divergence-class-only` flag allows phased ratchet per §11.4.50 so plan-data-cleanup and gate-enforcement can proceed independently.